Class Method vs Static Method



KEY IDEAS

	Instance	@classmethod	@staticmethod
1st arg	`self`	`cls`	none
Instance state	yes	no	no
Class state	via `type(self)`	yes	no
Called on	instance	class / instance	class / instance
Typical use	behaviour of one object	factories, class-wide state	pure utilities

TIP Decision: need `self`? **instance method**. Need only the class (factory, class state)? **classmethod**, which also works for subclasses because `cls` is the subclass. Need neither? **staticmethod** (or a module-level function).

EXAMPLE

```python
class Pizza:
    count = 0

    def __init__(self, toppings):
        self.toppings = toppings
        Pizza.count += 1

    @classmethod
    def margherita(cls):              # factory using cls
        return cls(["mozzarella", "tomato"])

    @classmethod
    def made(cls):                    # reads class state
        return cls.count

    @staticmethod
    def cm_to_inch(cm):               # pure utility
        return cm / 2.54
```

```python
class BigPizza(Pizza):
    pass

p = BigPizza.margherita()
print(type(p).__name__)               # → BigPizza
print(Pizza.made())                   # → 1
print(round(Pizza.cm_to_inch(30), 1))  # → 11.8
```